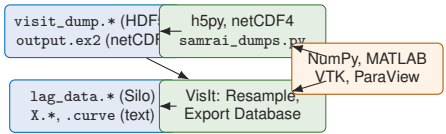
CONVERTING BETWEEN DATA TYPES; EXPORTING TO MATLAB AND PYTHON

What an IBAMR run writes and in which format; reading the fluid dumps, the finite-element files, the points and the text files in Python and MATLAB; VisIt's Resample and Export as the other route; PIV and other programs' output into VisIt



every reader on these slides was run on this series' data; the 2013–2020 decks and scripts credited where they are quoted

Laura A. Miller

Department of Mathematics, University of Arizona

Laura A. Miller used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. name every file an IBAMR run writes, its container (HDF5, netCDF, Silo, text) and what reads it (VisIt, Python, MATLAB), and set the input keys that control them;
2. read the fluid dumps straight into NumPy (`samrai_dumps.py`) or MATLAB (`samrai_dumps.m`), put the adaptive patches on a uniform grid, and check the result against VisIt;
3. do the same through VisIt, with the Resample operator and **Export Database** (the 2017–2018 recipe), scripted for a series of frames (`resample_export.py`), and know the five things that silently give an empty file in 3.5;
4. read the exported VTK, the ExodusII file of an IBFE run, the Silo points and the HDF5 state dumps with the readers of `examples/analysis`, in Python and in MATLAB/Octave;
5. compute a quantity VisIt does not give you directly, the flow rate across a line against time, from the dumps (the 2020 lecture's `flappy_ends.m`, redone on IB/02 and IB/11);
6. bring other programs' fields into VisIt (PIV components to VTK, COMSOL's VTU) and know where ParaView, Tecplot and the LCS tools fit.

> Everything shown was run: the readers on IB/13, IB/02, IB/11, IBFE/01, IBFE/10 and an upstream run, with Python 3.12 (h5py 3.10, netCDF4 1.6, vtk 9.1, meshio 5.3) and Octave 8.4 (netcdf 1.0.17) in WSL; the `.m` files were checked in Octave where Octave has the function (netCDF yes, `h5read` no), and `samrai_dumps.m` later in MATLAB R2024a (Appendix A).

WHERE THIS CHAPTER COMES FROM

- "Taking a 2D slice from 3D IBAMR with Visit" (2017, 17 slides) and "6 – Resampling AMR grids to Uniform Grid, exporting to .vtk, and recording in Python" (2018, 21 slides): the Resample operator, the plane of one sample, **Export Database** to VTK, the `2D_Slicer.py` recording with its time loop, Nick's MATLAB reader, the `mopenpiv` reader, the "cut off the edges" warning, the KillDevil commands. Quoted and redone in 3.5 (slides 14–21).
- "Converting IBAMR output to Matlab and analyzing it" (MATH 769, 23 April 2013, 12 slides) and "5 – Reading forces and converting IBAMR output to Matlab and analyzing it" (2020, 24 slides): Eric Tytell's `importsamrai.m` and its `V` structure, the `OUTPUT_DIV_U` workaround, `flappy_ends.m` and `vol_flow_rate.m`, `pcolor_frames.m`; the Lagrangian half is chapter 11, the Eulerian half is here (slides 12, 34–38).
- Christopher Strickland's 2017 scripts and README ("IBAMR → VTK → NumPy"): `read_IBAMR3d_py27.py` and `read_vtk_data.py`; Nick Battista's 2017 MATLAB readers `read_Structured_Vector_Field_Data_From_vtk.m` and `Convert_vtk2LCS.m`; the lab's 2017 Read Me notes on batch processing (Lindsay Waldrop), LCS options, Dabiri's LCS code and Arvind's circulation script; the 2017 "Read Me and To Do" whose item "overview of translating one file type to another (.visit, .m, .py, .txt, .vtk, etc)" was the one-slide stub this chapter fills.
- The 2019 "6. Analyze PIV Data" document (DaVis batch processing; the lost `toVTK.py`) and the 2019 COMSOL lecture's slides 12–14 (export as VTU, open in VisIt).
- New here: the file formats read from the files themselves (`h5py`, `ncdump`); `samrai_dumps.py/.m`, `read_exodus.py/.m`, `read_vtk_legacy.py/.m`, `hier_data_h5.py`, `resample_export.py`, `flow_rate.py`, `piv_to_vtk.py` in `examples/analysis`, each tested; the comparison of the Python route with VisIt's; what VisIt 3.5 does wrong headless.

THE MAP: WHAT A RUN WRITES, AND WHAT READS IT (1)

file	written by	container	readers
`viz_IB2d/dumps.visit`, `visit_dump.NNNNN/`	SAMRAI's VisIt writer, every `viz_dump_interval`	HDF5 (`summary.samrai` + one `processor_cluster` file per rank)	VisIt (SAMRAI plugin); Python `h5py` via `samrai_dumps.py`; MATLAB `h5read` via `samrai_dumps.m`; not ParaView's own readers
`viz_IB2d/lag_data.visit`, `lag_data.cycle_NNNNNN/`	`LSiloDataWriter` (chapter 28)	Silo, PDB driver (not HDF5)	VisIt; export from VisIt as VTK; the `X.*` files hold the same positions
`viz_IB2d/output.ex2` (one per part)	libMesh's `ExodusII_IO` (IBFE)	ExodusII = netCDF-4/HDF5	VisIt, ParaView; Python `netCDF4` via `read_exodus.py`; MATLAB `ncread` via `read_exodus.m`
`hier_data_IB2d/X.NNNNN`, `F.NNNNN`	the examples' `output_data` (chapter 11)	PETSc ASCII text	`hier_data.py`, `read_hier_data.m`

THE MAP (2): CURVES, VTK, RESTART FILES, AND THE KEYS

file	written by	container	readers
`hier_data_IB2d/ hier_data.NNNNN. samrai.PPPPP`	SAMRAI's HDF5 database (upstream ex1, the IBFE and complex-fluid examples)	HDF5	`hier_data_h5.py`
`*.curve`	VisIt (queries over time, picks, lineouts)	text	`importdata`, `numpy` (chapter 28)
`*.vtk`, `*.vtr`	VisIt's **Export Database**; IB2d's own output	VTK legacy or XML	VisIt, ParaView; `read_vtk_legacy.py/.m`, the `vtk` module, `pyvista`
`restart_IB2d/`	SAMRAI + `writeFEData ToRestartFile`	HDF5 + libMesh xda	the program only

The input keys: `viz_writer`, `viz_dump_interval`, `viz_dump_dirname`, `data_dump_interval`, `data_dump_dirname`, `restart_dump_interval` in `Main` (chapter 7); `output_U`, `output_P`, `output_Omega`, `output_Div_U` in the integrator block choose the fluid fields.

THE FLUID DUMPS ON DISK

```
viz_IB2d/                              # IB/13 human, 4 ranks, 102
  frames
  dumps.visit                          # visit_dump.00000/summary.
  samrai ...
  visit_dump.19096/                    # frame 56: the time step
  number
    summary.samrai                     # 20 kB: the hierarchy and
  the extents
    processor_cluster.00000.samrai     # 115 kB: rank 0's patches
  and fields
    processor_cluster.00001.samrai     # 114 kB
    processor_cluster.00002.samrai     #  99 kB
    processor_cluster.00003.samrai     #  99 kB
```

```
$ file visit_dump.19096/summary.samrai
Hierarchical Data Format (version 5) data
$ h5ls -r visit_dump.19096/summary.samrai | head
    # hdf5-tools
$ python3 -c "import h5py; f = h5py.File('summary
  .samrai'); f.visit(print)"
```

- A frame is a folder named by the time step, holding one HDF5 file per MPI rank with the data and one summary file with the layout; `dumps.visit` lists the summary files in order (chapter 26). The same for 3D (`viz_IB3d/`).
- They are HDF5 files whatever their extension: `h5ls`, `h5dump` (apt `hdf5-tools`) and `h5py` open them; nothing in them is private to VisIt.
- The values are single precision (`float32`) and nodal for `U`, `P`, `Omega` (IBAMR 0.12 on), cell-centred for `Div U`: these are VisIt's copies, not the solver's arrays. The solver's own double-precision state is the `hier_data.*.samrai.*` dump of slide 29, when a program writes it.
- Sizes: the human carotid run's 102 frames take 49 MB; the 512^2 channel's 51 frames 38 MB; a 3D run grows fast, which is why one dumps every few hundred steps and not every step (chapter 7's interval table).

INSIDE SUMMARY.SAMRAI

```
BASIC_INFO/
  number_dimensions_of_problem  [2]
  var_names             [b'U', b'P', b'Omega', b'Div U']
  var_cell_centered     [0, 0, 0, 1]        # 0 = nodal
  var_number_components [2, 1, 1, 1]        # U has U.00, U.01
  var_number_ghosts     [[0,0,0], ...]
  number_levels         [2]
  number_patches_at_level [4, 6]            # 10 patches in all
  number_processors     [4]   number_file_clusters [4]
  dx                    [[0.125, 0.125, 0], [0.03125, 0.03125, 0]]
  XLO                   [0, 0, 0]
  ratios_to_coarser_levels [[-1,-1,0], [4,4,0]]
  time [0.9548]   time_step_number [19096]   grid_type [b'CARTESIAN']
extents/
  patch_extents  (10,) lower, upper, xlo, xup     # per patch
     e.g. lower (0,0) upper (12,23) xlo (0,0) xup (1.625,3.0)
  patch_map      (10,) processor, file cluster, level, patch number
  Omega-Extents  (10,) data_is_defined, min, max, ...  # per patch, per
   variable (two more flags)
```

- Everything the reader needs: the variables and their centering, the levels and their spacings, the refinement ratios, and for every patch its index box (`lower`, `upper`, inclusive, in the cells of its own level), its physical box, and which rank's file holds it.
- The level-0 patches tile the domain: their `upper` maxima plus one give the coarse cell count (50×24 here), and the ratios the finest (200×96).
- The per-variable extents are the file's own minimum and maximum: a check for any reader (`Omega` runs from -3272.4 to 3474.7 at this frame; the reader reproduces both).
- The 2013 lecture's `OUTPUT_DIV_U = FALSE` workaround ("the space in the name") is not needed: `Div U` is an ordinary dataset name to HDF5.

INSIDE THE PROCESSOR_CLUSTER FILES

```
processor.00000/
  level.00000/
    patch.00000/
      Div U   (312,)  >f4      # 13 x 24 cells
      Omega   (350,)  >f4      # 14 x 25 nodes
      P       (350,)  >f4
      U.00    (350,)  >f4      # the x component
      U.01    (350,)  >f4
  level.00001/
    patch.00000/   Div U (3120,)  Omega (3233,) ...
    patch.00001/   Div U (480,)   Omega (549,)  ...
```

```
a = g["Omega"][()]                    # flat, x fastest (SAMRAI's
  order)
ncell = upper - lower + 1             # (13, 24)
A = a.reshape((ncell + 1)[::-1])      # nodal: (25, 14) = [y, x]
```

- Each rank's file holds only its own patches, grouped by level and patch number; the patch's index box is in the summary file (slide 7); a cell-centred variable has $n_x n_y$ values, a nodal one $(n_x + 1)(n_y + 1)$ ($13 \times 24 = 312$ and $14 \times 25 = 350$ for patch 0).
- The arrays are flat with x varying fastest, so `reshape((ny, nx))` gives an array indexed `[y, x]`, the orientation `pcolormesh` and MATLAB's `meshgrid` expect.
- A vector is stored as components `U.00`, `U.01` (`U.02` in 3D); VisIt reassembles `U` from them, a reader does the same.
- `>f4`: big-endian single precision (HDF5 converts on reading); the values are the nodal interpolants VisIt plots, not the staggered-grid velocities of the solver.
- Ghost cells: `var_number_ghosts` is zero for these files, so the arrays cover exactly the patch; the `hier_data` HDF5 dumps of slide 29 record a ghost width per patch (`d_ghosts`), zero in the dumps read there.

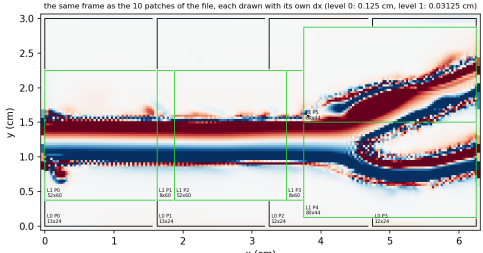
SAMRAI_DUMPS.PY: THE READER

```
import samrai_dumps as sd
d = sd.Dump("viz_IB2d/visit_dump.19096")      # one frame
d.time, d.levels, d.patches_per_level         # 0.9548, 2, [4, 6]
d.variables                                   # ['U', 'P', 'Omega',
  'Div U']
x, y, omega = d.uniform("Omega")              # finest grid: x
  (201,), y (97,), omega (97, 201)
x, y, (u, v) = d.uniform("U")                 # a vector: one array
  per component
x, y, divu = d.uniform("Div U")               # cell-centred: x
  (200,), y (96,)
d.patch_extent_values("Omega")                # ([-3272.36],
  [3474.73]) from the summary
for step, t, folder in sd.frames("viz_IB2d"): # the series from
  dumps.visit
    d = sd.Dump(folder); ...; d.close()
```

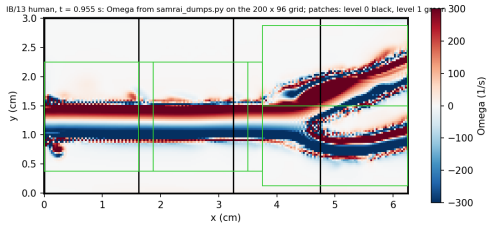
```
$ python3 samrai_dumps.py viz_IB2d/visit_dump.19096 Omega
time 0.9548 step 19096  levels 2  patches [4, 6]  finest cells [200,
   96]  dx [0.03125, 0.03125]
  U      nodal  components 2  file extents ([-30.02, -65.59],
  [110.97, 52.80])
  P      nodal  components 1  file extents ([-1748.2], [14797.5])
  Omega  nodal  components 1  file extents ([-3272.36], [3474.73])
  Div U  cell   components 1  file extents ([-0.1426], [0.3614])
  Omega[0] on the uniform grid: shape (97, 201), min -3272.36 max
  3474.73, NaN 0
```

- `examples/analysis/samrai_dumps.py`: 220 lines, `numpy` and `h5py` only. `Dump` reads the summary, opens the rank files as needed, `patch_data(patch, var)` gives one patch's arrays, `uniform(var, level)` the whole domain on one level's grid, `frames(viz_dir)` the list from `dumps.visit` with the times.
- `uniform` paints the levels coarse to fine: a coarse patch is interpolated onto the fine nodes (bilinear) or repeated per cell (cell-centred data), and a finer patch overwrites it wherever it exists. Nodal data come back on the nodes of the finest level ($N + 1$ per direction), cell data on the centres (N).
- The arrays are `[y, x]` (`[z, y, x]` in 3D, filled the same way: trilinear for nodal data, repeated for cell data); `nan` marks nothing here, because the level-0 patches cover the box.
- `line_flux(x, y, u, x0, ylo, yhi)` integrates a field across a vertical line (slide 34).

FROM PATCHES TO A UNIFORM GRID



the ten patches of frame 56 as the file holds them, each drawn with its own spacing (level 0 black, level 1 green)



`d.uniform("Omega")`: the same frame on the 200×96 grid of level 1, the patch boxes drawn over it

- The AMR hierarchy is a set of rectangles; where a level-1 patch exists its nodes are the fine grid's nodes, and elsewhere the level-0 values are interpolated to them: nothing is invented, the picture is smooth across the patch edges because the coarse data are linear across them.
- A uniform array is what every analysis wants: a flux across a line, a time average at a point, a Fourier transform in x, a comparison with PIV on its regular grid, a MATLAB `pcolor`. The price is the memory of the finest grid everywhere (200×96 here; 512^2 for the channel; a 3D run's finest grid can be larger than the machine wants, so take a plane with `uniform(..., level=k)` on a coarser level or VisIt's Resample with a box).

CHECKED AGAINST VISIT

- VisIt's **Resample** operator on the same frame, with the sample points put exactly on the fine nodes (201×97 over $[0, 6.25] \times [0, 3]$), exported as VTK and read back: the two `Omega` arrays agree to 0.012 at worst on a range of 6747 (1.8×10^{-6} relative, rms 3×10^{-4}); 18 of the $19\,497$ nodes differ by more than 10^{-6} of $\max|\Omega|$ (five by more than 10^{-6} of the range), most of them within four cells of the inlet boundary $x = 0$, where VisIt's interpolation is its own. The reader reproduces the file's extents exactly; VisIt's resample is within 2×10^{-3} of them.
- `U` the same way (VisIt's export of the vector: components agree to 6×10^{-5}, single precision).
- So the Python route and the VisIt route are the same numbers; choose by convenience: Python when the analysis is a loop over frames or a quantity VisIt has no query for, VisIt when a picture or one of its queries is enough (chapter 29).
- Chapter 16's convergence study, chapter 24's exercise (the average of T) and chapter 9b's pump measures (the kinetic energy of the flow) are the kind of thing the reader is for.

> The upstream `ex1` that the examples' `output_data` descends from also wrote the Eulerian fields as HDF5 (`hier_data.NNNNN.samrai.PPPPP`, slide 29); this series' examples dropped that, because the VisIt dumps already hold the fields and this reader opens them.

THE MATLAB VERSION, AND TYTELL'S IMPORTSAMRAI.M (2013) (1/2): THIS SERIES' READER

```
% examples/analysis/samrai_dumps.m (MATLAB, h5read)
d = samrai_dumps('viz_IB2d/visit_dump.19096');          % the
  layout
d = samrai_dumps('viz_IB2d/visit_dump.19096', 'Omega'); % + d.x,
  d.y, d.data (ny x nx)
pcolor(d.x, d.y, d.data); shading flat; axis equal
d = samrai_dumps(folder, 'U');                          % d.data
  {1}, d.data{2}
```

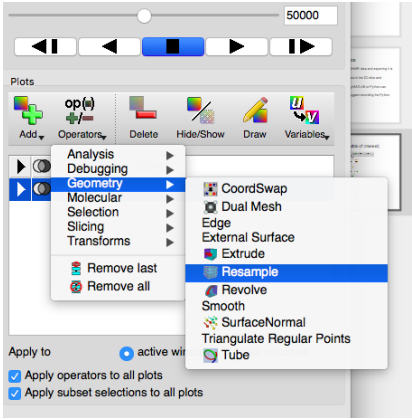
- `samrai_dumps.m` mirrors the Python reader line by line with `h5read` (MATLAB's HDF5 reader; it returns the arrays transposed, which the code undoes), `kron` for cell data and `interp2` for nodal data. It was written against the Python output (Octave 8 has no `h5read`); run later in MATLAB R2024a on a dump of IB/03, after two fixes (`h5read` returns the variable names as strings; a variable named `cell` hid the function `cell`), it gives the same arrays as the Python reader (Appendix A). 2D only.

THE MATLAB VERSION, AND TYTELL'S IMPORTSAMRAI.M (2013) (2/2): THE 2013 TOOLBOX

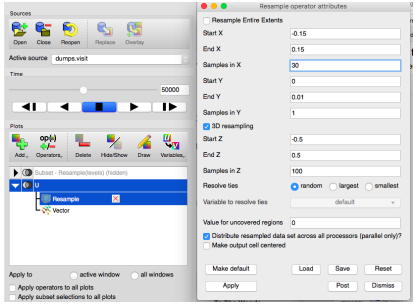
```
% the 2013 / 2020 lecture (E. Tytell's toolbox, matlab-tytell/CFD,
    after install_here.m)
[x,y,Vinterp,V] = importsamrai(pathname,'interpolaten',[GridSizex
    GridSizey])
% V(k).level_number, .xlo, .xup, .rows, .cols: one element per
  patch
% V(k).U_0, .U_1, .P, .F0, .F1, .Omega: the fields of that patch
% Vinterp: the same interpolated to the GridSizex x GridSizey
  grid
```

- The 2013 lecture used Eric Tytell's `importsamrai.m`: "imports samrai data from files in the directory pathname. Files should be named `summary.samrai` and `processor_clusterXXXXX.samrai`"; its output `V` is "a structure containing one element for each grid region" with `level_number` "from 1 (coarsest) usually to 4 (finest)", `xlo`, `xup`, `rows`, `cols` and the fields, and `Vinterp` the interpolated data. The same idea; its fields are the old names (`U_0`, `F0`) and it needed `OUTPUT_DIV_U = FALSE` "due to the space in the name". The toolbox is third-party (link in chapter 11) and not shipped.
- `Forceread.m` and `positionread.m` of the same lecture are chapter 11's `read_hier_data.m`.

VISIT'S ROUTE: THE RESAMPLE OPERATOR (2017–2018)



Operators → Geometry → Resample
(2017/2018 decks, VisIt 2.7.1)



the attributes: a plane of $30 \times 1 \times 100$ samples at $y \in [0, 0.01]$

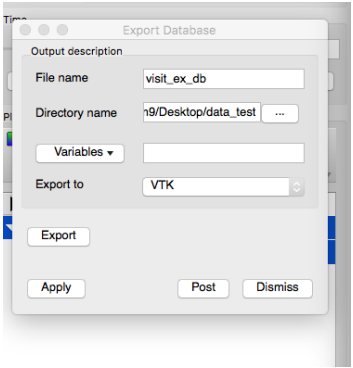
1. Open `dumps.visit`, plot `U` (or the variable of interest).
2. "Unclick apply operators to all plots. Unclick apply subset selections to all plots."
3. Highlight the plot, **Operators → Geometry → Resample**; double-click it.
4. "Unclick resample entire extents"; set the samples and the start/end of each direction; **Apply**.

Windows: "6 – Resampling AMR grids" (2018; the same images as the 2017 deck), VisIt 2.7.1 on a Mac.

WHAT THE RESAMPLE OPERATOR DOES

- "The main issue with taking a 2D slice from 3D IBAMR data and exporting it is that it is on an adaptive grid. We can handle this in Visit by resampling the data in the 2D slice and exporting it to vtk" (2017). Resample puts the plotted variable on a regular grid of `samplesX` $\times$ `samplesY` $\times$ `samplesZ` points between the start and end values (`useExtents = 1` takes the whole domain); `defaultValue` fills points no patch covers; `distributedResample` shares the work among a parallel engine's processes (on, VisIt 3.5.0 exported each state in pieces written into one file, so `resample_export.py` turns it off; chapter 31); `cellCenteredOutput` makes cells instead of nodes.
- The 3.5 window has the same fields (**Resample Entire Extents**, **Start X** . . . , **Samples in Z**, **3D resampling**, **Resolve ties**, **Value for uncovered regions**, **Distribute resampled data set**, **Make output cell centered**).

EXPORT THE RESAMPLED DATA: **FILE → EXPORT DATABASE**



the Export window (2018): name, directory, variables, **Export to: VTK**

- "Highlight Resample. Go to File → Export database. Choose your file name and directory. Export to VTK. If you choose no variables, it will just export U. Click export." (2018). The export writes the active plot's data *after its operators*: with Resample on it, a regular grid (VTK `RECTILINEAR_GRID`); the variables added in the window come along, resampled too.
- "You can also export .ex2 files to vtk" (chapter 28 did it: the mesh and `dX`).
- The warning that still holds: "When you resample, if the grid is very coarse in places (especially the edges), those values may be set to zero. The solution when doing your analysis is to cut off the edges of the domain." (`defaultValue` is what you see there.)
- The result "has the vector U = (Ux, Uy, Uz) in the (x, y, z) plane"; the 2017 deck's example was a plane of 30×100 points through a 3D run, "use only U = (Ux, Uz) in the (x, z) plane".

Window: "6 – Resampling AMR grids" (2018; the same image as the 2017 deck), VisIt 2.7.1 on a Mac.

FIVE WAYS VISIT 3.5 GIVES YOU AN EMPTY EXPORT, HEADLESS (1)

All found while scripting the recipe on this series' runs (`wsl/final/ch33_*.sh`); none shows an error.

1. The file name and directory are ignored: every export writes `.vtk` (no name) into the *current directory* of the VisIt process, and **Export all time states** overwrites it state after state. Rename the file after each export (`resample_export.py` does); the GUI on a desktop may behave, chapter 28 reported the same for the Exodus export.
2. A SAMRAI database reports zero extents in its metadata (`GetMetaData(db).GetMeshes(0).minSpatialExtents` is all zeros), so a resample box built from them is a point and the file holds zeros: ask the drawn plot instead (`Query("SpatialExtents")`), or type the box.
3. A plane of zero thickness exports zeros: `samplesY = 1` with `startY = endY` resamples nothing; the 2017 deck's `startY = 0, endY = 0.01` was this fix (a slab one per cent of the axis thick works; two samples work too).

FIVE WAYS VISIT 3.5 GIVES YOU AN EMPTY EXPORT (2), AND A READER THAT CANNOT READ THEM

4. In one session, a Vector plot exported a broken grid (coordinate lists of one entry) and a Pseudocolor of the automatic `U_magnitude` expression exported `U` as zeros; a later re-test of both exported correctly (VisIt only warns that the Vector plot "yielded no data"), so treat these two as possible, not certain. Plot a Pseudocolor of a scalar (`U[0]` as an expression) and add `U` in the variable list: that route exported correctly every time.
5. An export without Resample (the AMR mesh itself) creates a named directory and a `.visit` list, but headless 3.5.0 writes every patch file as `<cwd>/.vtk`, each overwriting the last, and the list points at `<dir>/.vtk`: nothing can read the result (reproduced twice). Resample first, or move each patch file yourself.

> `meshio` 5.3 cannot read VisIt's legacy VTK files (their `FIELD` block has a string entry, `MeshName`: "`'string'`") and does not know `.vtr`; the `vtk` module and `read_vtk_legacy.py` read them, `pyvista` reads `.vtr`.

RESAMPLE_EXPORT.PY: THE RECIPE SCRIPTED FOR A SERIES OF FRAMES

```
$ visit -nowin -cli -s resample_export.py -- viz_IB2d/dumps.visit U out nx=201 ny=97
   states=56
state 56  t = 0.9548  ->  out/U_0056.vtk
   (ok; plot U_c0 -- Min = -30.02 ... Max = 110.97 ...)
$ visit -nowin -cli -s resample_export.py -- viz_IB3d/dumps.visit U plane \
      nx=141 ny=1 nz=101 x0=-4 x1=10 y0=0 z0=-5 z1=5 states=all
```

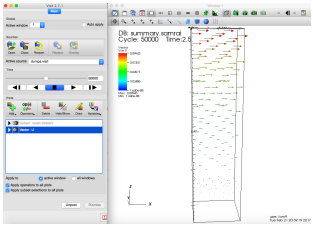
```
OpenDatabase(db); DeleteAllPlots()                    # the automatic 'levels' Subset
  plot goes
DefineScalarExpression("U_c0", "U[0]"); AddPlot("Pseudocolor", "U_c0"); DrawPlots()
Query("SpatialExtents"); ext = GetQueryOutputValue()  # the box (the metadata's is
  zero)
AddOperator("Resample"); r = ResampleAttributes(); r.useExtents = 0
r.startX, r.endX, r.samplesX = x0, x1, nx; ...; r.is3D = 1; r.defaultValue = 0
SetOperatorOptions(r)
e = ExportDBAttributes(); e.db_type = "VTK"; e.db_type_fullname = "VTK_1.0"
e.dirname = out; e.variables = ("U",); opts = GetExportOptions("VTK")
for s in states:
    SetTimeSliderState(s); DrawPlots(); e.filename = "U_%04d" % s
    ExportDatabase(e, opts)                           # writes <cwd>/.vtk ...
    os.replace(".vtk", os.path.join(out, "U_%04d.vtk" % s))   # ... so rename it
```

- Left: how the script is called, and what it prints per state (the plot's **MinMax**, so that an empty export shows up at once); below it, the lines that matter, from the file in `examples/analysis`. The next slide says what the script is and where it comes from.

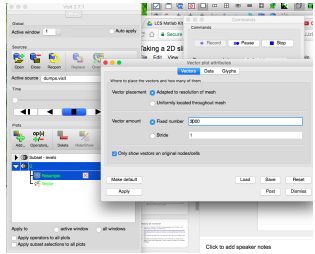
RESAMPLE_EXPORT.PY: WHAT IT IS, AND ITS ANCESTORS (2017)

- `examples/analysis/resample_export.py` runs inside VisIt's Python (`visit -nowin -cli -s`, chapter 32): the database, the variable, the output folder, the samples, the box, the states, legacy ASCII or binary output. It prints the plot's **MinMax** at each state as a check that the data are there.
- Its ancestors: the 2017 `2D_Slicer.py` (the recording of the GUI steps with `TimeSliderNextState()` in a `while` loop, `ExportDBAtts.filename = "visit_ex_db_%s" % k`) and Christopher Strickland's `read_IBAMR3d_py27.py` (2017: Python 2.7, `sys.path.append` to VisIt 2.11's `site-packages`, `LaunchNowin()`, 11^3 samples over the full extents by default, files `vtk3d/<run>/IBAMR_db_NNN.vtk`, and a note on fixing `frontend.py` on Windows). VisIt 3 embeds Python 3 and none of the path tricks is needed.
- The recording itself (**Controls** $\rightarrow$ **Command**, Record) is chapter 32; the 2018 deck's slides 15–21 (KillDevil modules, `visit -nowin -cli -s script.py`) are chapters 31–32.

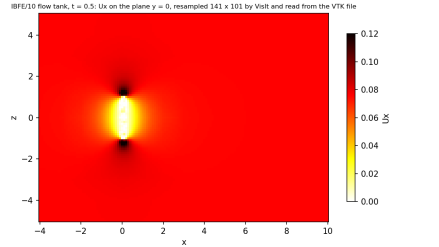
A 2D PLANE OUT OF A 3D RUN



2017: the resampled plane's 30×100 vectors in the 3D view (a tower of coral, `sample_viz_IB3d`)



"Calculate the number of vectors you resampled. In this case it's 30x1x100=3000. Use that for the number of vectors."



now: IBFE/10's flow tank, U_x on $y = 0$ at $t = 0.5$, 141×101 samples, exported by `resample_export.py` and read with `read_vtk_legacy.py`

- "You can't actually pick a 2D slice, but you can export only one value of x, y or z": one sample in the normal direction (with the thin slab of slide 17) is the plane; the exported file is a `RECTILINEAR_GRID` of `DIMENSIONS 141 1 101` with the vector at each point and `TIME` in its field data.

Left two: "Taking a 2D slice from 3D IBAMR" (2017), VisIt 2.7.1.

THE SAME PLANE FROM THE DUMPS, AND WHEN TO USE WHICH

- The same plane straight from the dumps: `samrai_dumps.py` in 3D fills the finest grid of the whole box (trilinear for nodal data), from which one slice is taken; for a 256^3 run that is 16 M nodes per variable, so VisIt's resample of the plane alone is the lighter tool there.

READING VTK IN PYTHON: THREE WAYS

```
# vtk DataFile Version 4.2           <- what VisIt wrote (legacy ASCII)
vtk output
ASCII
DATASET RECTILINEAR_GRID
FIELD FieldData 3
MeshName 1 1 string
amr_mesh
CYCLE 1 1 int
19096
TIME 1 1 double
0.9548
DIMENSIONS 201 97 1
X_COORDINATES 201 double
0 0.03125 0.0625 ...
Y_COORDINATES 97 double
...
POINT_DATA 19497
SCALARS Omega double
LOOKUP_TABLE default
...                                  # one value per point, x fastest
```

```
import read_vtk_legacy as rv                  # numpy only (examples/
  analysis)
v = rv.read_vtk("Omega_0056.vtk"); x, y, z = rv.grid_axes(v)
omega = v["data"]["Omega"]                    # (97, 201); vectors keep
  a last axis of 3
t = v["field"]["TIME"][0]
```

- No library: `read_vtk_legacy.py` parses the legacy ASCII file into axes, point data (reshaped `[y, x]`), points and cells for the unstructured kinds (an exported Exodus mesh, Silo points), and the `FIELD` block (`TIME`, `CYCLE`).

READING VTK IN PYTHON (2): THE VTK MODULE, PYVISTA, MESHIO

- The `vtk` module (Strickland 2017, `read_vtk_data.py`): `vtkRectilinearGridReader` / `vtkStructuredPointsReader` / `vtkUnstructuredGridReader`, `numpy_support.vtk_to_numpy`, the `dataset_adapter` for `FieldData['TIME']`; reads binary legacy and, with the XML readers, `.vtr`. "Because the vtk library is used to load the vtk file data, these functions should not be sensitive to line numbers or slight variation in format" (his README). Indexed `[z, y, x]`.
- `pyvista` (`pyvista.read("file.vtr")`) when it is installed; `meshio` for files VisIt did not write (slide 17).
- The 2017 decks pointed at `mopenpiv/vtk_tools.py` ("read_vtk_scalar and read_vtk_vector starting at line 127") on Bitbucket, which is gone.

READING VTK IN MATLAB

```
% examples/analysis/read_vtk_legacy.m (checked in Octave 8.4)
v = read_vtk_legacy('Omega_0056.vtk');
% v.type 'RECTILINEAR_GRID', v.dims [201 97 1], v.x, v.y, v.z
% v.data.Omega (97 x 201), v.field.TIME
pcolor(v.x, v.y, v.data.Omega); shading flat; axis equal
w = read_vtk_legacy('U_0056.vtk');           % w.data.U (97 x 201
   x 3)
quiver(w.x, w.y, w.data.U(:,:,1), w.data.U(:,:,2))
```

```
% N. Battista, 2017: read_Structured_Vector_Field_Data_From_vtk.m
strName = 'visit_ex_db';                % the file, without .vtk
[U,V,W,X,Y,Z,Nx,Ny,Nz] = give_Me_The_Data_Please(strName);
% skips 5 header lines, sscanf('%*s %f %f %f %*s') for DIMENSIONS,

% fscanf the X, Y, Z coordinates, then 3*Nx*Ny*Nz numbers,
% reshape(U, Ny, Nx, Nz)
```

- `read_vtk_legacy.m` reads the same headers as the Python version (`RECTILINEAR_GRID`, `STRUCTURED_POINTS`, `UNSTRUCTURED_GRID`, `POLYDATA`; scalars and vectors; the `FIELD` block) and returns arrays indexed `(y, x)`; ASCII only (convert a binary export with VisIt's **Legacy Ascii** option, or read it in Python).
- Nick Battista's 2017 reader is written for one layout, the `RECTILINEAR_GRID` that VisIt 2.x exported from a 3D resample, with the line numbers of that file built in: it fails on a file with a `FIELD` block in front (VisIt 3 writes one) and on scalar exports (the `LOOKUP_TABLE` line). Its `Convert_vtk2LCS.m` loops over `visit_ex_db_<k>.vtk`, reads each with the same function and writes `x z u w` text lines for Dabiri's LCS code ("Loop assumes we are looking at velocity data in the x,z-plane"). A note in it: the Z loop reads `Nz` values with a format built for `Ny` (a copy slip; harmless, because `fscanf` recycles the format).
- The 2013 lecture's `pcolor_frames.m` drew `Vinterp` this way; slide 38 does it in Python.

THE EXODUSII FILE IN PYTHON AND MATLAB

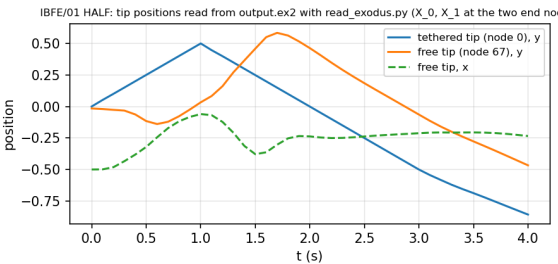
```
$ ncdump -h viz_IB2d/output.ex2            # netcdf-bin; IBFE/01
  HALF
dimensions: time_step = UNLIMITED ; // (41 currently)
            num_dim = 2 ; num_nodes = 101 ; num_elem = 130 ;
            num_el_blk = 1 ; num_nod_per_el1 = 3 ; num_nod_var =
  8 ;
variables:  double time_whole(time_step) ;
            double coordx(num_nodes) ; double coordy(num_nodes) ;
            int connect1(num_el_in_blk1, num_nod_per_el1) ;  //
  elem_type = "TRI3"
            char name_nod_var(num_nod_var, len_name) ;
            double vals_nod_var1(time_step, num_nodes) ; ...
  vals_nod_var8
```

```
import read_exodus as rx                    # netCDF4
m = rx.Exodus("viz_IB2d/output.ex2")
m.nodal_names        # ['dX_0','dX_1','X_0','X_1','F_0','F_1','
  U_0','U_1']
m.var("dX_1", 10)[0] # 0.500402: node 0 at t = 1 (chapter 28's
  PickByNode, HALF run)
m.positions(10)      # coords + dX: equals (X_0, X_1) to 1e-16
m.connect["1"]       # (130, 3) 0-based triangles
```

- ExodusII is netCDF with a fixed vocabulary: `coordx`, `coordy`, one `connect<b>` per element block, the names in `name_nod_var` (fixed-width characters), the fields as `vals_nod_var<k>(time_step, num_nodes)`. `ncdump -h` shows it all; `file` calls it HDF5 because netCDF-4 is HDF5 underneath (so `h5py` opens it too, without the names' meaning).

THE EXODUSII FILE (2): MATLAB, AND THE PLATE'S TIPS

```
ex = read_exodus('viz_IB2d/output.ex2');      % ncread; Octave:
  pkg load netcdf
a = ex.var('dX_1', 11);  a(1)                 % 0.500402 (1-based
  state and node)
p = ex.positions(11);  tri = ex.connect1;     % triplot(tri, p(:,1)
  , p(:,2))
```



- `read_exodus.py` (Python, `netCDF4`) and `read_exodus.m` (MATLAB's `ncread`; Octave's `netcdf` package, `apt install octave-netcdf`) give the times, the reference coordinates, the connectivity, any nodal field at one state or all, and the current configuration. Checked against chapter 28's `PickByNode` on the HALF run (both languages); the Pick printed in chapter 28 is the FULL run (0.500316).
- The tips of the HALF plate from the file (right): the tethered tip follows the prescribed path, the free tip swings behind it; the README's numbers of chapter 17 came from this.

THE SILO POINTS: THROUGH VISIT, OR FROM THE X FILES

```
# VisIt CLI: the point mesh of frame 56 as a VTK file
OpenDatabase("viz_IB2d/lag_data.visit"); DeleteAllPlots()
AddPlot("Mesh", "human_vertices"); DrawPlots();
  SetTimeSliderState(56); DrawPlots()
e = ExportDBAttributes(); e.db_type = "VTK"; e.db_type_fullname =
   "VTK_1.0"
ExportDatabase(e, GetExportOptions("VTK"))          # -> <cwd>/.
  vtk, rename it
```

```
v = rv.read_vtk("points_0056.vtk")                  #
  UNSTRUCTURED_GRID / POLYDATA
pts = v["points"][:, :2]                            # (1045, 2)
X = hd.read_vec("hier_data_IB2d/X.19096")           # chapter 11's
  reader, same step
abs(pts - X[:, :2]).max()                           # 1.0e-5 cm:
  the Silo file's single precision
```

- The `lag_data` files are Silo with the PDB driver (`DBCreate(..., DB_PDB)` in `LSiloDataWriter.cpp`): not HDF5, so `h5py` refuses them, and there is no Silo reader in a plain Python. VisIt reads them, and **Export Database** with the Mesh plot active writes the points (and the spring edges from `<name>_mesh`) as VTK, one state at a time (slide 17).
- Simpler: the `X.NNNNN` text files of chapter 11 hold the same positions in double precision at the `data_dump_interval`; with equal intervals (the inputs of this series) the step numbers match frame for frame, as the check at left shows (the difference is the Silo file's single precision). Forces are only there.
- Silo's own Python module comes with the Silo library when it is built with Python support (VisIt 3.5.0's Linux tarball does not include it); chapter 32 uses it if a reader without VisIt is ever needed.

THE HDF5 STATE DUMPS OF THE UPSTREAM EXAMPLES

```
PatchHierarchy/                                    # hier_data.00033.
  samrai.00000
  d_number_levels [3]
  level_0000/  d_boxes, d_ratio_to_coarser_level, d_number_patches ...
    level_0000-patch_0000/
      INSStaggeredHierarchyIntegrator::P##INSStaggeredHierarchyIntegrator
  ::CURRENT/
        d_box, d_ghost_box, d_ghosts [0 0], d_depth [1]
        d_data/d_array (512,) >f8               # cell data, 8 x 64: no
  ghost cells in these dumps
      INSStaggeredHierarchyIntegrator::U##INSStaggeredHierarchyIntegrator
  ::CURRENT/
        d_data0/d_array (520,)  d_data1/d_array   # side data: x faces, y
   faces
      PolymericStressForcing::C_cc##AdvDiffSemiImplicitHierarchyIntegrator
  ::CURRENT/
        d_data/d_array, d_depth [3]              # a tensor, depth 3
```

```
import hier_data_h5 as hd5
f = hd5.HierData("hier_data_IB2d/hier_data.00033.samrai.00000")
f.variables                          # the three names above
for p in f.patches(f.variables[0]):
    p["level"], p["lo"], p["hi"], p["data"]      # P: [y, x] without
  ghosts
# U: p["data"] = [u on the x faces (ny, nx+1), v on the y faces (ny+1, nx)
  ]
```

- IBAMR's own `examples/IB/explicit/ex1` (chapter 11's writer descends from it), the IBFE examples and the complex-fluids ones write, every `data_dump_interval`, the solver's state through SAMRAI's HDF5 database: one file per rank, the hierarchy, and per patch every registered variable with its context (`##CURRENT`), its box, its ghost width and its depth.

THE HDF5 STATE DUMPS, CONTINUED

- These are the real arrays: double precision, `P` cell-centred, `U` side-centred (the x component on the x faces: `(ny, nx+1)`), with `d_ghosts` ghost cells around them when the writer adds any (these dumps have none: `d_ghosts [0 0]`, and $512 = 8 \times 64$). `hier_data_h5.py` strips the ghosts, if present, and returns each patch; the output above is from the complex-fluids `ex2` run of chapter 25 (16 patches on 3 levels; the polymeric stress tensor as a depth-3 field).
- Reading them is the way to check a solver quantity VisIt's nodal copies blur (`Div U` at the cell centres, the staggered velocity); the series' examples do not write them (slide 11).

CURVES, AND THE LISTS AFTER A RESTART

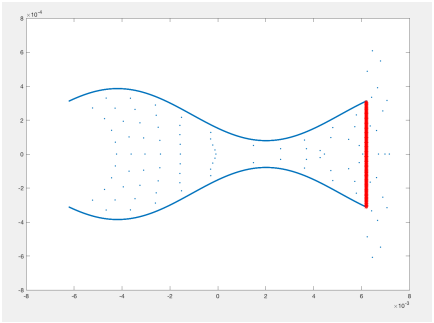
```
C = importdata('plate_dX_queries.curve');  % one block per curve
  (chapter 28)
t = C.data(:,1); y = C.data(:,2);
```

```
# chapter 3: after a restart, dumps.visit lists
  only the new frames
$ wc -l dumps.visit.after_restart dumps.visit
    # 35 vs 77 in the test run
$ ls -d visit_dump.* | sort -t. -k2 -n | sed 's#$
  #/summary.samrai#' > dumps.visit
$ for d in $(ls -d lag_data.cycle_* | sort -t_ -
  k3 -n); do echo "$d/$d.summary.silo"; done >
  lag_data.visit
```

```
frames = sd.frames("viz_IB2d")        # reads dumps.visit:
  rebuild it first
```

- `.curve` files (queries over time, picks with a time curve, lineouts) are text with a # header per curve; chapter 29's `read_curve.m` and `read_curve.py` read them (chapter 28 shows `importdata` and a short Python loop); Lindsay Waldrop's 2017 note on batch processing adds that they "are pretty easy to import into R".
- A restarted run truncates `dumps.visit` and `lag_data.visit` to the frames after the restart (chapter 3's test: 35 lines for 77 folders on disk); the readers of this chapter trust the lists (`sd.frames` walks `dumps.visit`), so rebuild them with chapter 3's two lines before an analysis spanning the restart. The Exodus file of an IBFE run is one file and `append(from_restart)` keeps it whole (chapter 17).
- The frame times are in the files (`BASIC_INFO/time`, `time_whole`), not in the lists; `sd.frames` returns them.

THE 2020 LECTURE'S EULERIAN ANALYSIS: `flappy_ends.m` AND `vol_flow_rate.m`



"The figure that is generated shows the tube (blue) and the line over which the volumetric flux is calculated (red)."

- "This function imports SAMRAI data into Matlab, interpolates to a uniform grid, reads in the lagrangian positions, and calculates the volumetric flow rate and maximum velocity at the end of the tube" (`flappy_end_analysis`, 2020). Its inputs were typed at the top of the script: `GridSizex = 4096`, `GridSizey = 512` ("size of the finest grid"), `starting_time = 10000`, `time_step = 5000` ("this should match the interval in the input2d file"), `final_time = 55000`, `timestep = 7.5e-6` ("dt used in IBAMR"), `Lminx = -0.01 ...`, `pperitop = 5080` ("the number of points along the top of the tube").

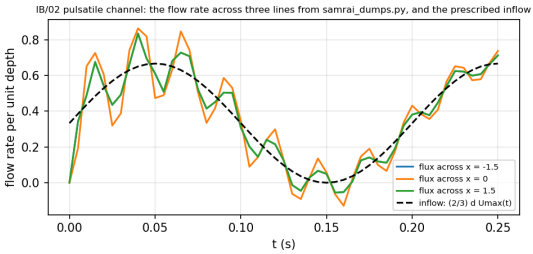
"5 – Reading forces and converting IBAMR output to Matlab" (2020), slides 15–20 and the figure of slide 17.

THE 2020 LECTURE'S FLOW-RATE FUNCTION, AND WHAT REPLACES ITS HAND-TYPED INPUTS

- `vol_flow_rate.m` "returns the volumetric flow rate across the end of the tube and the maximum velocity along the end of the tube at the given instance in time", with the row and column indices it used; the run was "a long heart tube with contraction prescribed as a traveling sine wave"; "the volumetric flow rate in m^3/s" is, in 2D, a rate per unit depth.
- The pattern, grid size and interval copied by hand from the input file, is what `samrai_dumps.py` reads from the files instead (slide 34).

THE FLOW RATE ACROSS A LINE, NOW: FLOW_RATE.PY

```
$ python3 flow_rate.py viz_IB2d x=-1.5,0.0,1.5 y=-0.5:0.5 out=flux
  .csv plot=flux.png
time,x=-1.5,x=0,x=1.5
0,0,0,0
0.005,0.3433,0.1929,0.3429
...
0.25,0.7125,0.7374,0.7126
# mean over the frames: x=-1.5: 0.3726, x=0: 0.3730, x=1.5:
  0.3726
```

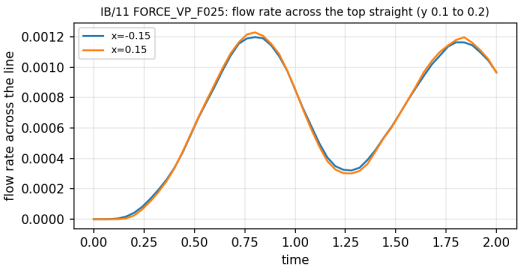


- `flow_rate.py` walks `dumps.visit`, puts `U` on the finest grid at each frame and integrates u across each line (trapezoid rule after a linear interpolation in x; `line_flux` in `samrai_dumps.py`); horizontal lines take v.

THE FLOW RATE ACROSS A LINE: WHAT THE NUMBERS SAY

- IB/02, the pulsatile channel (chapter 7's 0.25 s run, 51 frames, 512^2 finest grid): the flow rates across $x = -1.5$ and $x = 1.5$ agree with one another to 0.0017 over the whole run and to 0.0004 after 0.15 s (0.05 % of the peak 0.83): incompressibility, checked. The line at $x = 0$ differs from them by up to 0.16 (0.077 after 0.15 s, 9 % of the peak; the listing shows it): it crosses the refined region around the channel, and the cause was not run down. Report the lines you trust and say why.
- Against the prescribed inflow, $Q_{\mathrm{in}} = \frac{2}{3} d\, U_{\max}(t)$ with $U_{\max} = \frac{1}{2} + \frac{1}{2}\sin(2\pi\, 5\, t)$ (the parabola of chapter 7's `gcoef` integrated over $|y| < d/2$): a start-up transient (the run begins at rest with the inflow already at $Q = 1/3$), then the outer lines within 0.07 (8 % of the peak) after 0.15 s; the point walls are not sealed, and the box around the channel is open. The means over the 51 frames, 0.3726 ($x = \pm 1.5$) against 0.3781; over frames 1–50 ($t > 0$), 0.380 against 0.379.

THE FLOW RATE OF THE VALVELESS PUMP (IB/11)



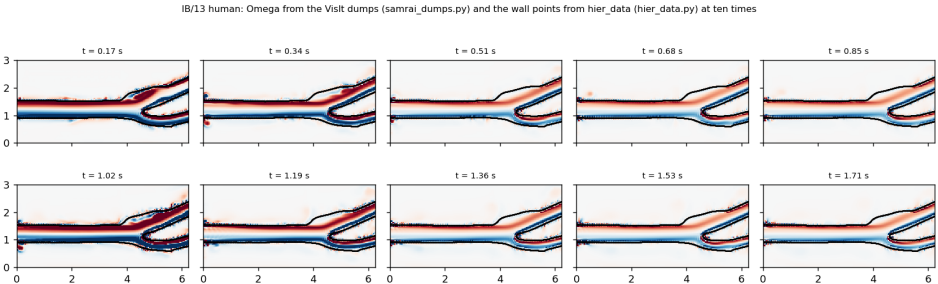
IB/11 `FORCE_VP_F025` (chapter 9b's shipped suction run): the flow rate across the loop's top straight (the rigid side opposite the elastic section) at $x = -0.15$ and $x = 0.15$ m, $y \in [0.1, 0.2]$, over two periods (51 frames).

- The two lines give the same curve (means 6.64 and 6.65×10^{-4} m^2/s): the flux along the tube is one number at each instant.
- The mean, 6.6×10^{-4} m^2/s per unit depth through a 0.1 m tube, is a mean speed of 6.6 mm/s in the $+x$ direction along the top, so the loop turns clockwise and the flow along the elastic section is in $-x$ (chapter 29 finds the same flux on both bends and across the whole pericardium), pumped by a squeeze that never closes the tube: Liebau's valveless pumping, measured as a flux. Chapter 9b's README measured the markers' drift in the elastic section, $+1.1$ and $+1.0$ cm per period: against the net flow there. A drift of marker points is not a flux.

THE PUMP'S FLOW RATE (2): WHERE TO PUT THE LINE

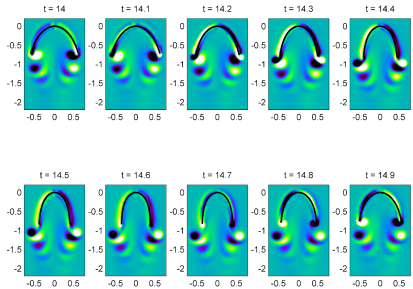
- A line across the lower, elastic section at $x = -0.15$, between the walls' rest positions, gave almost nothing, probably because the tube moves, bulging past such a line in places and narrowing inside it in others, so the line crosses part of the tube and part of the gap between tube and pericardium. Across the whole pericardium (y from -0.245 to -0.055) the flux is the loop's again, 6.6×10^{-4} m^2/s in $-x$ (chapter 29); check the geometry before trusting a line (chapter 9b's figures show where the tube is).
- The same loop over `sd.frames` gives any other quantity: the maximum speed on the line, the mean pressure drop between two lines, the kinetic energy in a box.

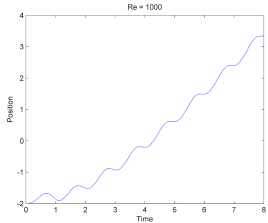
FRAMES AND POSITIONS TOGETHER: `PCOLOR_FRAMES.M` IN PYTHON



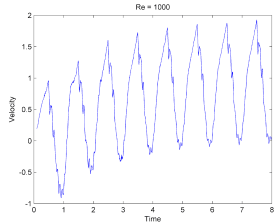
- The 2013 lecture's `pcolor_frames.m`, in the words of its header (spelling corrected): reads positions from `hier_data` and vorticities from the `viz_IB2d` SAMRAI data and makes a 5×2 plot of vorticity at 10 times; it is specific to jellyfish swimming, in that the y position of the top of the bell is read and all y values are shifted so that the top of the bell is always at $y = 0$. Its output and the position and velocity plots of `readpositions.m` are three of the four figure slides of that lecture (slide 39).
- The same in 30 lines (`tools/ch33_frames.py`): `sd.frames`, `d.uniform("Omega")` and `pcolormesh` for the field, `hier_data.read_vec("X.<step>")` (chapter 11) for the points of the same step, a 2×5 grid of axes.

THE 2013 LECTURE'S FIGURES





"Plots generated from readpositions.m": the bell's position, $\mathrm{Re} = 1000$



its velocity against time

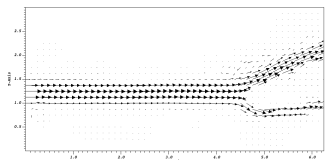
"Plots generated by pcolor_frames": a jellyfish bell at $t = 14$ to 14.9

- The position and velocity of the bell's top against time, read from the `X` files and differentiated: chapter 11's `read_series` does the reading, `numpy.gradient` the rest; chapter 10's IB/10 bell is the modern run.
- `Force_vs_time.m` of the same lecture ("the force on each boundary point is summed", rescaled by ρSU^2) is chapter 11's `leaf_forces.py`.

PIV INTO VISIT: `PIV_TO_VTK.PY`

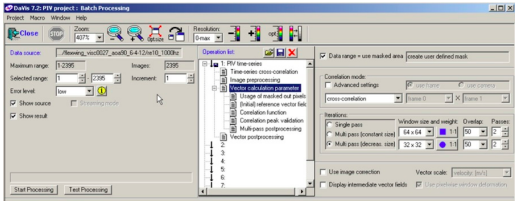
```
$ python3 piv_to_vtk.py Vx.txt Vy.txt piv_field.vtk --dx 0.125 --
  dy 0.125 --flip
wrote piv_field.vtk: 51 x 25 vectors, spacing 0.125 x 0.125, |U|
  up to 109.8
```

```
# vtk DataFile Version 3.0
PIV vector field
ASCII
DATASET STRUCTURED_POINTS
DIMENSIONS 51 25 1
ORIGIN 0 0 0
SPACING 0.125 0.125 1
POINT_DATA 1275
VECTORS U double
0.0 0.0 0.0 ...
```

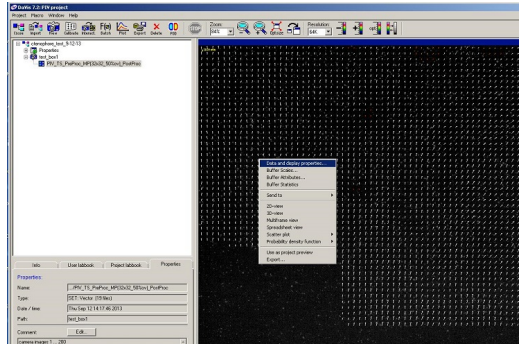


- The 2019 PIV document (old notes credited to "Steve"): in DaVis, "extract scalar field: vector component" for V_x and V_y "separately one at a time", export each frame as ASCII text, then "the python script toVTK.py (found in scripts/forPIV)" turns the two files into a VTK file "which can then be opened in VisIT". That script is lost; `piv_to_vtk.py` is its replacement: two text arrays (rows $= y$, columns $= x$; DaVis, OpenPIV or NumPy), the spacing and origin, `--flip` when the first row is the top of the image.
- The output is a `STRUCTURED_POINTS` vector field, the format IB2d writes too: VisIt's Vector and Pseudocolor plots, `magnitude(U)`, `U[0]`, and the LCS operator (FTLE is one of its outputs) take it as they take an IBAMR field; the picture is a test field (IB/13's velocity subsampled every fourth node) drawn by VisIt from the file.
- `write_vtk_vector(path, u, v, dx, dy)` in the same file writes any gridded field you compute in Python for VisIt.

PIV PROCESSING AS THE 2019 DOCUMENT SHOWED IT



DaVis 7.2: **F(batch)**, “PIV – time series” with a multi-pass of decreasing window size, 64×64 to 32×32



the project window with a vector field below its image set; “Data and display properties” to scale the vectors

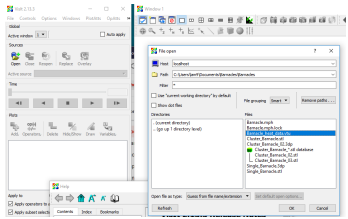
- The document’s recipe: highlight the image set, **F(batch)**, choose “PIV – time series” (“no pre-processing, no post-processing, and a multi-pass of decreasing size, 64x64 – 32x32”), “choose the range of images you want and the increment . . . such that the particles move a few pixels with each image step”, start processing; then scale the display (“Data and display properties”, vector length, vector colour range).

“6. Analyze PIV Data” (2019 lab document), DaVis 7.2 screenshots.

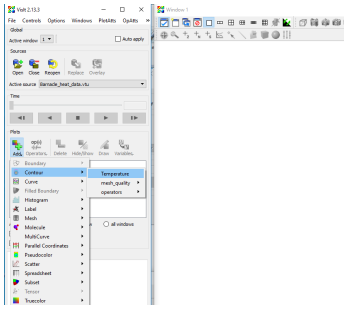
PIV PROCESSING (2): WHAT THE DOCUMENT LEFT TO DO, AND OPENPIV

- Its to-do list (“write a tutorial on how to use the MATLAB script on the PIV computer to export to .mat files”, “find the files that convert ASCII text to .vtk from Steve”) is what the previous slide closes; the comparison of PIV fields with IBAMR fields (averages, circulation, LCS) is chapter 29.
- OpenPIV (the 2017 instructions by Julia Samson, E11; the `mopenpiv` fork is gone) is the open route to the vector fields; DaVis the commercial one (its batch window on the previous slide).

OTHER PROGRAMS' OUTPUT INTO VISIT: COMSOL'S VTU



VisIt 2.13.3 (Windows) opening `Barnacle_heat_data.vtu`, the file COMSOL exported



Add → **Contour** → **Temperature** → **Draw** on it

- The 2019 "COMSOL work flow" lecture solved a heat problem in COMSOL and brought the result into VisIt: in COMSOL, **Results** → **Export** → **Data**, type **Unstructured *.vtu** (the lecture: "Export -> Data 1 -> Unstructured *.vtu"; "not the easiest to find"); in VisIt, **Open** the `.vtu` and plot. COMSOL itself is outside this series (chapter 1b lists it); its export is the point here: any program that writes VTK, XML VTK (`.vtu`, `.vtr`, `.vts`), Exodus, Silo, netCDF, HDF5 with a known layout, or plain CSV points can be looked at next to an IBAMR run in the same VisIt window, with the same plots and queries.

"Lecture 8: COMSOL work flow and remote simulations" (2019; its COMSOL log is dated 19 February 2019), slides 13–14; the VisIt windows only.

PARAVIEW, TECPLOT, LCS AND CIRCULATION: POINTERS

- The reverse holds: VisIt's exports (VTK, Silo, Tecplot, STL, PLY, Xmdv) are the way into those programs (slide 16).
- ParaView (https://www.paraview.org) opens VTK, XML VTK, Exodus and netCDF natively and, through the VisIt bridge (a ParaView built with it; Silo only when built with the Silo library), SAMRAI dumps and Silo; the lab's 2014 ParaView tutorial PDF and 2021 recording are third-party and listed only. Everything in Part V has a ParaView equivalent (Resample To Image, Slice, Plot Over Line, Python shell); this series chose VisIt because IBAMR's output is written for it.
- Tecplot: the 2017 Read Me notes that Dabiri's LCS code writes "a data file type for Tecplot" and that "Tecplot is like commercial Visit"; VisIt exports Tecplot files (**Export Database** → **Tecplot**).
- LCS and FTLE: the 2017 "LCS options" slides list VisIt's own **LCS** operator, with FTLE among its outputs (`OperatorPlugins()` has `LCS`, no separate FTLE), MIT's and ETH's (George Haller's) MATLAB codes; the DabiriLCS Read Me points at Shadden's tutorial and at `Convert_vtk2LCS.m` (slide 25) for feeding exported fields to it. Chapter 29 runs the VisIt operator.
- Circulation: Arvind's note (2016) lists what his MATLAB script needs from a 2D field, "x, y, u, v, du/dx, du/dy, dv/dx, dv/dy and z-vorticity"; the note writes the vorticity as $w_z = \partial v/\partial y - \partial u/\partial x$, which is a slip for $\partial v/\partial x - \partial u/\partial y$. The easy version in the same Read Me, "make a box around the vortex and calculate average vorticity", is a VisIt query (chapter 29); from a uniform array it is `omega[box].mean()`.
- The 2017 to-do that became this chapter: "Overview of translating one file type to another (.visit, .m, .py, .txt, .vtk, etc)": the map of slide 4.

EXAMPLES/ANALYSIS: THE READERS, AND WHICH CHAPTER USES EACH

file	what it reads or writes	chapter
`hier_data.py`, `read_hier_data.m`, `read_input.m`	the `X.*`, `F.*` text files; input-file constants	11
`leaf_forces.py`, `leaf_forces.m`	the drag of IB/03 from the forces (worked example)	11
`samrai_dumps.py`, `samrai_dumps.m`	the VisIt dumps (HDF5) onto a uniform grid; frames; line fluxes	33
`flow_rate.py`	the flow rate across lines against time, from the dumps	33
`resample_export.py`	VisIt CLI: Resample + Export Database over a series of frames	33, 32
`read_vtk_legacy.py`, `read_vtk_legacy.m`	legacy ASCII VTK (VisIt's exports, IB2d's output)	33
`read_exodus.py`, `read_exodus.m`	the ExodusII file of an IBFE run	33, 28
`hier_data_h5.py`	the HDF5 state dumps of the upstream examples	33
`piv_to_vtk.py`	two text arrays to a VTK vector field (and any gridded field)	33, 29

All Python 3 with `numpy`; `h5py`, `netCDF4` and `vtk` where the format needs them (`apt install python3-h5py python3-netcdf4 python3-vtk9` on Ubuntu, `pip install h5py netCDF4 vtk` elsewhere; VisIt's own Python has none of them, so the VisIt scripts stay inside VisIt and the NumPy scripts outside). The `.m` files were checked in Octave 8.4, and `samrai_dumps.m` (no `h5read` in Octave) in MATLAB R2024a (Appendix A).

THEN AND NOW

the decks and scripts said (2013–2020)	now
“Download the matlab-tytell.zip file . . . run install_here.m” (2013)	`samrai_dumps.m` / `.py` in `examples/analysis`, no toolbox
“It is necessary to not output Div U when running IBAMR due to the space in the name”	not necessary: `Div U` is an ordinary HDF5 dataset name
`V.U_0`, `V.U_1`, `V.F0`, `V.F1` (the 2013 field names)	`U.00`, `U.01` in the files; no `F` field in the VisIt dumps of this series’ examples (`output_F` off)
`GridSizex = 4096` typed into the script	read from `summary.samrai` (`number_patches`, `dx`, `ratios`)
“record your python commands in visit 2.5.2” for KillDevil	VisIt 3.5 everywhere, Python 3 inside it (chapter 32)
`sys.path.append("C:/Program Files/LLNL/VisIt 2.11.0/lib/site-packages")`, `frontend.py` edited on Windows	`visit -cli` runs the script in VisIt’s own Python; no path or hotfix
`ExportDBAtts.filename = "visit_ex_db_%s" % k`	the name is ignored headless in 3.5: rename `.vtk` after each export
“If you choose no variables, it will just export U”	true for a plot of `U`; add the variables you want, resampled too
Nick’s reader “uses only U = (Ux, Uz) in the (x, z) plane”, line numbers built in	`read_vtk_legacy.m/.py` read the headers; the `vtk` module reads anything
`mopenpiv/vtk_tools.py` on Bitbucket; `toVTK.py` in `scripts/forPIV`	both gone; `read_vtk_legacy.py`, `piv_to_vtk.py`
“Dylan will start overview, Julia will add stuff on techplot” (the stub)	slide 4

WHAT CAN GO WRONG

symptom	cause and fix
`h5py` says “Unable to open file” on a `lag_data` file	Silo with the PDB driver: not HDF5; go through VisIt or the `X` files
the array from a patch looks transposed or scrambled	reshape as `(ny, nx)` (x fastest); nodal arrays are one larger per direction
the uniform array has NaNs	a level-0 patch missing from the list (a truncated `dumps.visit` is not it; a hand-edited summary is); an `IndexError` from the ratios table is what you get when `uniform` is asked for a level the dump does not have
the exported VTK file is empty, or all zeros	one of the traps of slides 17 and 18: the name, the metadata extents, the zero-thickness slab, the un-resampled multi-patch export, and (in one session) the Vector plot and the magnitude expression
`meshio.read` fails with `'string'`	VisIt’s `FIELD MeshName` entry; use `read_vtk_legacy`, the `vtk` module or `pyvista`
`ncread` is unknown in Octave	`pkg load netcdf` after `apt install octave-netcdf` or `pkg install -forge netcdf`
`h5read` is unknown in Octave	there is none in Octave 8; use the Python reader or MATLAB

WHAT CAN GO WRONG (2)

symptom	cause and fix
the frames begin at the restart step, the earlier ones missing	`dumps.visit` truncated to the frames after the restart: rebuild it (slide 31)
the flux across one line disagrees with another	the line crosses something besides the channel (a wall, a gap, a second structure), or the AMR level there is coarse: draw the line on the picture first
the flow rate differs from the prescribed inflow by a few per cent	the start-up transient; porous point walls; the open box: compare lines with each other first
VisIt's Python has no `numpy` or `h5py`	it is VisIt's own Python: export from VisIt, analyse outside, or install into VisIt's Python at your own risk

EXERCISES

1. Run `samrai_dumps.py` on a frame of IB/01 and check its `Omega` extents against VisIt's **MinMax** query; then plot the uniform array and the patch boxes as on slide 10.
2. Compute the kinetic energy $\frac{1}{2}\rho \int |\mathbf{u}|^2 \, dA$ of IB/02's box at every frame from the dumps and plot it against time; compare with the kinetic-energy measure of chapter 9b's pump.
3. Export the same frame with `resample_export.py` at half the finest spacing and at twice it; read both back and say where they differ from the Python array and why.
4. Take a plane out of IBFE/10 (`viz_IB3d`) at three heights and plot U_x with `read_vtk_legacy.py`; then do the same plane with `samrai_dumps.py`'s `uniform` and compare the two.
5. Read IBFE/08's two Exodus files with `read_exodus.py` and compute the gap between the beam's top nodes and the circle's bottom nodes against time (chapter 21's table of the gap at $t = 0, 1, 2, 4$).
6. Rebuild `dumps.visit` of the restart test of chapter 3 and show with `sd.frames` that the frames before the restart are back, with their times.
7. Make a PIV-like field from IB/13's velocity with `piv_to_vtk.py` at the spacing of a real PIV window (1 mm), open it in VisIt next to `dumps.visit`, and compare the two with a Pseudocolor of `magnitude(U)` and the same limits.
8. Chapter 24's exercise 6 asks for the average of T from the HDF5 state dump; do it from the VisIt dumps as well (`d.uniform("T")`) and check both against chapter 24's table and VisIt's **Average Value** query.

RULES OF THUMB

- Know the container: HDF5 (dumps, state dumps, netCDF-4 Exodus), Silo PDB (points), text (`X`, `F`, `.curve`); the extension says nothing.
- Python for loops over frames and quantities VisIt has no query for; VisIt for pictures and its queries; both give the same numbers.
- Nodal arrays are $N + 1$, cell arrays N; the files are x fastest; reshape as `(ny, nx)`.
- Check a reader against the file's own extents and against VisIt once, then trust it.
- Headless VisIt exports to `<cwd>/.vtk`: rename after every export; give the box yourself.
- Rebuild the `.visit` lists after a restart before any analysis spans it.
- Compare fluxes across two lines before comparing them with anything else.

SUMMARY

1. An IBAMR run's output is four formats: HDF5 for the fluid dumps (and the upstream examples' state dumps), Silo for the points, netCDF (ExodusII) for finite elements, text for the forces, positions and curves. All but Silo open in Python and MATLAB without VisIt.
2. `samrai_dumps.py` reads the dumps and assembles the adaptive patches on a uniform grid; VisIt's Resample + Export does the same, and the two agree to single precision.
3. VTK is the exchange format: VisIt writes it (with five headless traps), IB2d and PIV conversions write it, three readers read it; Exodus and `.curve` have readers of their own; the Silo points are the `X` files.
4. The 2013–2020 MATLAB analyses (flow rates, frames with positions) are a loop over frames with the reader; the examples' scripts in `examples/analysis` do them on IB/02, IB/11 and IB/13.
5. Other programs meet IBAMR in VisIt through their VTK or VTU exports; ParaView, Tecplot and the LCS tools take VisIt's exports.

Next: the IB2d chapters (4a–4f), then chapter 27 (3D), 29 (data analysis: queries, lineouts, LCS), 30 (movies), 31 (the cluster) and 32 (the command line and Python, where the recording of slide 19 is taught).

RECORDINGS AND SCREENSHOTS

- Shannon Jones's YouTube how-tos on recording to Python (`watch?v=RASHMJSgYM8`) and on running a script inside VisIt (`watch?v=BG7f6rvjHrw`), cited by the 2017–2018 decks and the 2017 Read Me; listed, not incorporated (chapter 32 covers the recording).
- The 2021 ParaView tutorial recording (`ParaviewTutorial2021.mp4`, third-party) and the COMSOL lecture's recordings are not among the front matter's course recordings; they stay on the Drive.
- The window screenshots are the 2017–2018 resampling decks' (VisIt 2.7.1 on a Mac; six of the 2018 deck's images are the 2017 ones), the 2019 PIV document's (DaVis 7.2) and the 2019 COMSOL lecture's two VisIt windows (2.13.3 on Windows); the plot pictures are new (Python and VisIt 3.5 in WSL). The re-shoot list in `plan/visit-screenshots-todo.md` has a chapter 33 section (the Resample window, the Export Database window with **Export all time states** and **Add variable**, both in 3.5).

REFERENCES AND CREDITS

- The course material: “Taking a 2D slice from 3D IBAMR with Visit” (2017) and “6 – Resampling AMR grids to Uniform Grid, exporting to .vtk, and recording in Python” (2018); “Converting IBAMR output to Matlab and analyzing it” (MATH 769, 23 April 2013) and “5 – Reading forces and converting IBAMR output to Matlab and analyzing it” (2020); “6. Analyze PIV Data” (2019); “Lecture 8: COMSOL work flow and remote simulations” (2019); the 2017 lab Read Me notes (“Read Me and To Do”, “LCS options”, DabiriLCS, circulation, batch processing).
- Christopher Strickland, `read_IBAMR3d_py27.py` and `read_vtk_data.py` with their README (2017); Nicholas A. Battista, `read_Structured_Vector_Field_Data_From_vtk.m` and `Convert_vtk2LCS.m` (2017, http://github.com/nickabattista); Eric Tytell’s `importsamrai.m` toolbox (2013; described, not shipped); Lindsay Waldrop’s batch-processing note (2017); Julia Samson’s OpenPIV instructions (2017); Arvind Santhanakrishnan’s circulation note (2016).
- SAMRAI’s VisIt data writer and HDF5 database: https://computing.llnl.gov/projects/samrai (`VisItDataWriter`, `HDFDatabase`); ExodusII: https://sandialabs.github.io/seacas-docs/; netCDF: https://www.unidata.ucar.edu/software/netcdf/; the VTK file formats: https://docs.vtk.org/en/latest/design_documents/VTKFileFormats.html; Silo: https://silo.readthedocs.io/; h5py, netCDF4-python, meshio, pyvista on PyPI; VisIt’s Resample operator and Export Database in the VisIt User Manual.
- Data: this series’ IB/13, IB/02, IB/11 (`FORCE_VP_F025`), IBFE/01, IBFE/10 runs and the upstream complex-fluids `ex2` run of chapter 25; scripts `wsl/final/ch33_*.sh`, `tools/ch33_figures.py`, `tools/ch33_frames.py`.